

Valutazione di pgvector come database unificato per architetture RAG

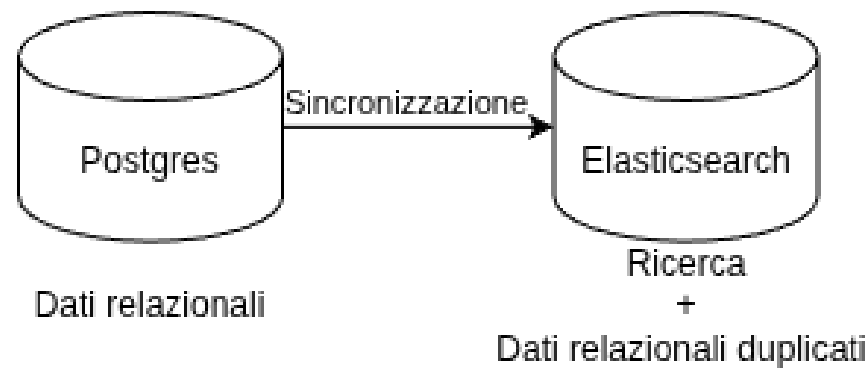
Davide Lorenzon

22 Settembre 2026



pat

ZUCCHETTI



Problema

Information retrieval in un contesto
relazionale

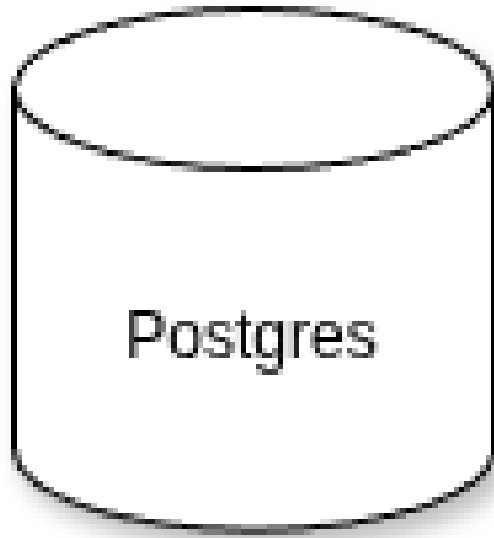
**Soluzione
attuale**

Postgres + Elasticsearch per la ricerca

Criticità

- **Overhead** di sincronizzazione e comunicazione
- **Join** lato backend

Pgvector



Dati relazionali + ricerca

Perché adesso

Avanzamento di
pgvector

Da valutare

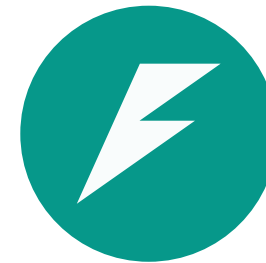
Fattibilità
tecnica e
performance

Estensione diretta e senza
altri DB esterni

Python



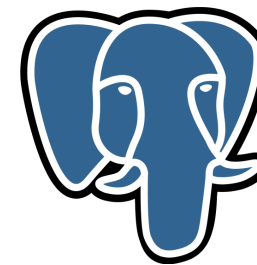
FastAPI

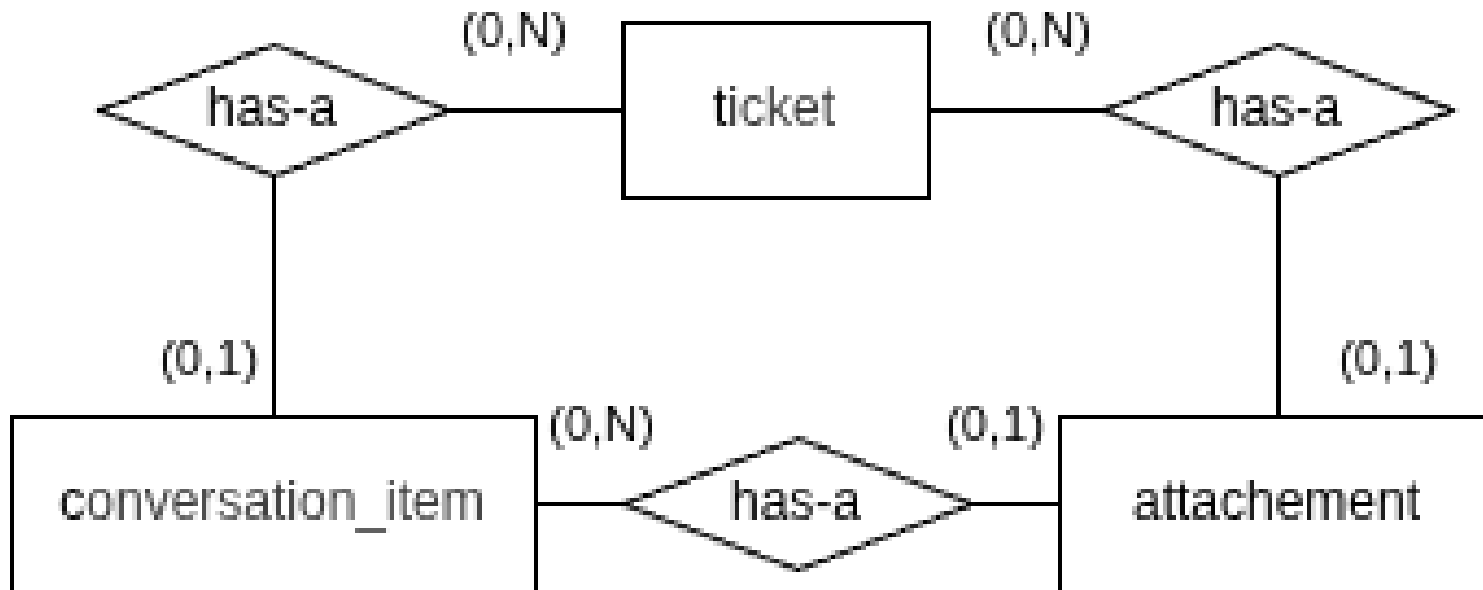


Grafana



full-text search nativa





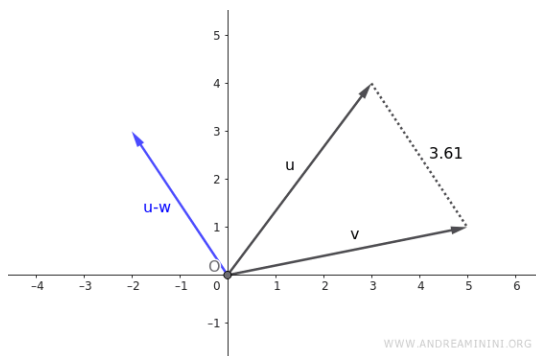
Interroga **tutte le entità** e
ricostruisce il contesto

Ricerca linked

È il caso d'uso che rende
Elasticsearch inadatto

Semantica

Similarità sui
vettori di
embedding



Full-text

Corrispondenza
lessicale



Ibrida

Fusione delle due
tramite **RRF**

```
-- Calculate RRF contribution
rfr AS (

  SELECT id,
    1.0 / (60 + rank) AS s FROM fulltext

  UNION ALL

  SELECT id,
    1.0 / (60 + rank) AS s FROM semantic
)

-- Sum the RRF scores, order by them,
-- and join back the original data

SELECT m.id, sum(s), m.description
FROM rfr

JOIN mock_items AS m USING (id)
GROUP BY m.id, m.description
ORDER BY sum(s) DESC
LIMIT 5;
```

Benefici attesi

- Stack **semplificato**
- **Linking** lato DB
- Integrazione più semplice su DB esistenti
- Proprietà **ACID**

Vincoli

- Solo **full-text nativo** (no dipendenze premature)
- **Pgvector**
- Testo ricercabile diviso in **chunk**
- Più campi ricercabili per entità

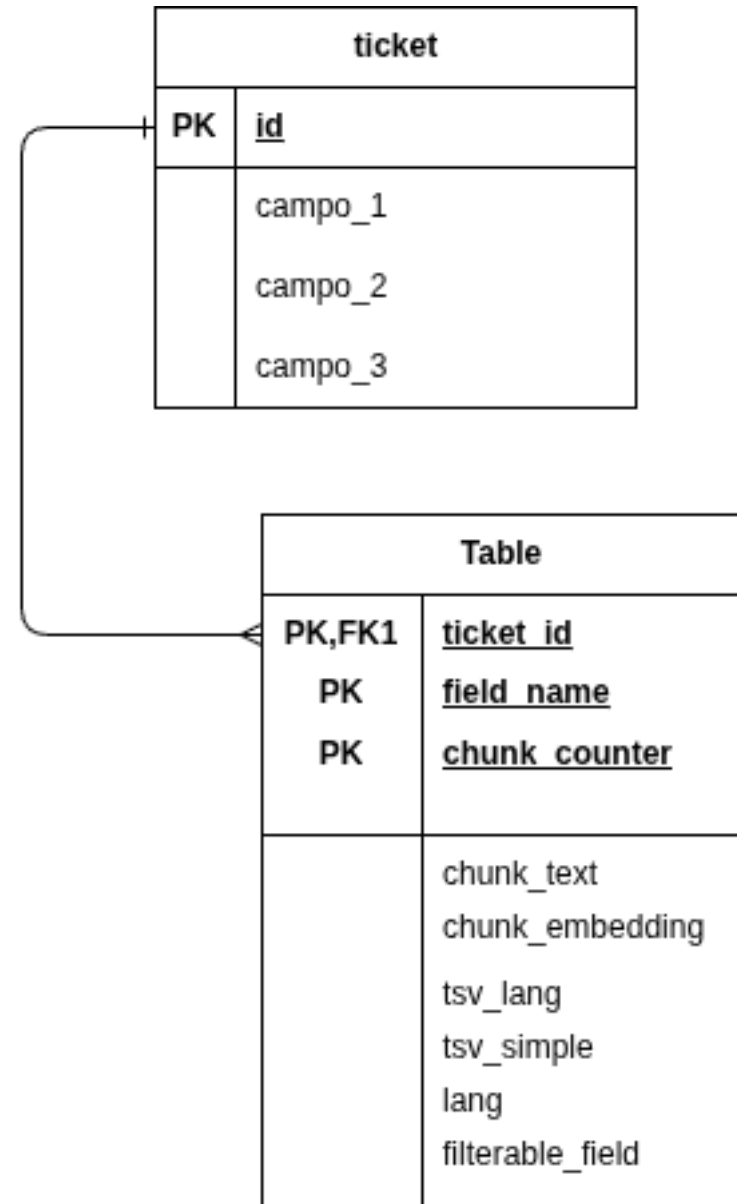
Tabella chunk di supporto

Partizionamento su
field_name

Indice **HNSW** su bit vector

Denormalizzazione dei campi
filterable

Oversampling e rescoring

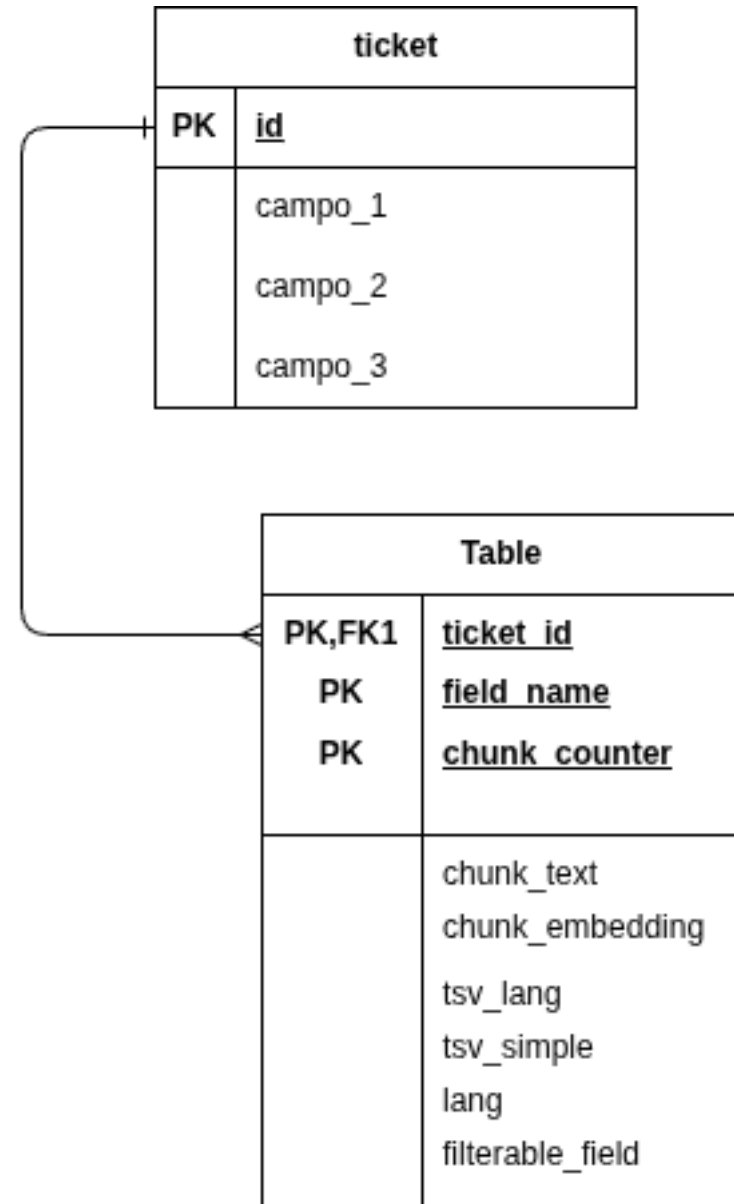


Non il focus principale ma
comunque necessaria alla
valutazione

Indice **GIN**

Punteggi sempre **comparabili**

Esplorati workaround



Locust

simula utenti paralleli

Metriche

hit rate, MRR, latenza

Grafana

dashboard collegata a Postgres

Ground truth

da dati sintetici





Dataset 10.000 Ticket	Conv. item 50.000	Attachment 60.000	Utenti paralleli test tramite Locust	Data visualization tramite dashboard Grafana
------------------------------------	---------------------------------------	-----------------------------	--	--

Anser Rate

ground truth nei risultati 100%

Latency

tempo tra richiesta-risposta
< 300ms

La ground truth è stata costruita automaticamente a partire dal dataset

ElasticSearch
+ join esterne



Postgres/pgvector
+ join integrate

Latency
200-300ms

- Linking all'interno della stessa query SQL: **nessun round-trip aggiuntivo**
- Ricerche tramite singola query: **database interrogato una sola volta**
- Il **linking relazionale è integrato** nella pipeline di retrieval

No overhead di sincronizzazione e join lato backend di ElasticSearch

Postgres

non ottimizzata
per lingua
5-10s

Postgres

ottimizzato per
lingua
~ 3s

Pgvector

ricerca semantica
< 300ms

- **Limite strutturale Postgres**
non offre le funzionalità dei motori full-text (BM25, ranking, filtering, Block MAX WAND)
- **Workaround:** Implementate funzioni di ranking e operazioni sugli array (overlap)
- **Conseguenza:** Ricerca full-text collo di bottiglia nella ricerca ibrida

Il limite è la full-text di PostgreSQL: valutare alternative come **pg_text_search** e **ParadeDB** (out of scope)

Risultati: cosa non ha funzionato





Grazie per l'attenzione!
Domande?